



Overview of Python as an Artificial Intelligence (AI/ML) language

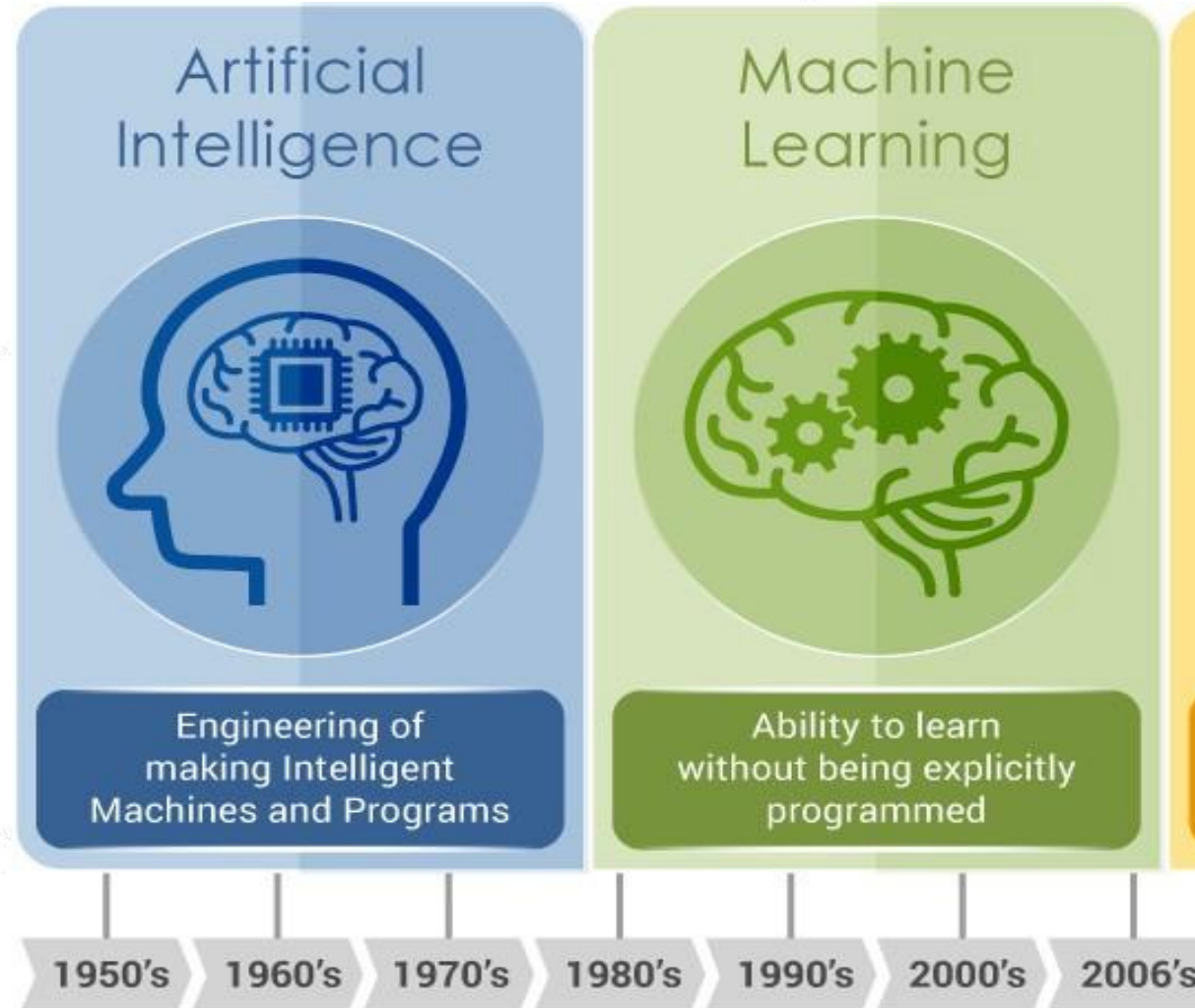


• Agenda

- OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE
 - INSTALLATION PYTHON AND EXECUTION OF PROGRAMMES
 - BASIC PYTHON PROGRAMMING: VARIABLES
-



OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE





OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

1. AI- artificial intelligence-

2. It is the science of training machines to perform human tasks
3. The term was invented in the 1950s when scientists began exploring how computers could solve problems on their own.
4. AI is a computer that is given human-like properties. Like our brain which works effortlessly and seamlessly to calculate the world around us.
5. Artificial Intelligence is the concept that a computer can do the same.
6. It can be said that AI is the large science that mimics human aptitudes.

1. Machine learning

2. It is a subset of AI that trains a machine how to learn
3. It is the art of study of algorithms that learn from examples and experiences.
4. Machine learning is based on the idea that there exist some patterns in the data which are to be identified and used for future predictions.
5. The machine does not need to be explicitly programmed by people. The programmers give some examples, and the computer is going to learn what to do from those samples.
6. The difference from hardcoding rules is that the machine learns on its own to find such rules.



OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

Why Python is Preferred for AI/ML

Python has become the go-to programming language for artificial intelligence (AI) development due to its simplicity and the powerful suite of libraries it offers

Its syntax is straightforward and closely resembles human language, which reduces the learning curve for developers and enables them to focus on solving AI problems rather than wrestling with complex coding issues. Python is highly favoured for AI and machine learning (ML) development for several compelling reasons that make it uniquely suitable for these technologies:

1) Extensive Libraries and Frameworks: Python boasts a wide range of libraries and frameworks, such as Scikit-learn for machine learning algorithms, TensorFlow, PyTorch, and Keras for deep learning, as well as NumPy, Pandas, and Seaborn for data manipulation and visualization. These libraries simplify coding tasks and reduce development time by providing pre-written code for common tasks.



OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

- 2)Ease of Learning and Syntax Simplicity: Python's syntax is straightforward, resembling everyday English, which significantly lowers the learning curve and enables developers to implement complex AI algorithms efficiently. The simplicity of Python syntax, which avoids brackets and emphasizes indentation, contributes to its ease of use and readability, making code less prone to errors and more maintainable .
- 3)No Need to Recompile Source Code: Python allows for dynamic modification and execution of code without the need for recompilation, offering flexibility and speeding up the development process. This feature is particularly advantageous in AI and ML projects, where iterative testing and tweaking are common.
- 4)Platform Independence: Python code can run on various operating systems, including Windows, Mac, UNIX, and Linux, without requiring any modifications, making it highly versatile for development across different platforms .
- 5)Strong Community Support: Python's large and active community contributes to a wealth of resources, including tutorials, forums, and documentation, which are invaluable for developers encountering challenges or seeking to enhance their knowledge and skills in AI and ML.



OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

Artificial Intelligence (AI) is a discipline that focuses on creating intelligent machines that can perform tasks that typically require human intelligence, such as visual perception, speech recognition, decision-making, and natural language processing. It involves the development of algorithms and systems that can reason, learn, and make decisions based on input data.

Essential Python Libraries for AI/ML

For anyone diving into Artificial Intelligence (AI) using Python, a handful of libraries make the development process significantly smoother and more efficient. These libraries, equipped with pre-built functions and tools, are essentials in the AI developer's toolkit:

NumPy: Fundamental for scientific computing in Python, NumPy offers comprehensive mathematical functions, random number generators, linear algebra routines, Fourier transforms, and more. It's particularly useful for handling large, multi-dimensional arrays and matrices, which are prevalent in AI tasks.

Pandas: A library offering high-level data structures and tools designed to make data analysis fast and easy in Python. Pandas are ideal for data munging and preparation and can be used in conjunction with other data analysis workflows in Python.



OVERVIEW OF PYTHON AS AN AI/ML LANGUAGE

Matplotlib: This plotting library allows for the creation of static, animated, and interactive visualizations in Python. Visual data representation is crucial in AI for data exploration and the presentation of results.

TensorFlow: An open-source library developed by the Google Brain team, TensorFlow is used for numerical computation using data flow graphs. It's particularly known for its applications in deep learning and neural networks.

PyTorch: Developed by Facebook's AI Research lab, PyTorch is a library for machine learning that provides great flexibility and speed while working with deep learning projects. It's known for its ease of use and efficiency in creating complex AI models.





PYTHON INSTALLATION (OFFLINE MODE)

Step 1: Select Version to Install Python

Visit the official page for Python <https://www.python.org/downloads/> on the Windows operating system. Locate a reliable version of Python 3, preferably version 3.10.11, which was used in testing this tutorial. Choose the correct link for your device from the options provided: either Windows installer (64-bit) or Windows installer (32-bit) and proceed .

STEP 2:Download the executable file.

Once you have downloaded the installer, open the .exe file, such as python-3.10.11-amd64.exe, by double-clicking it to launch the Python installer. Choose the option to Install the launcher for all users by checking the corresponding checkbox, so that all users of the computer can access the Python launcher application.Enable users to run Python from the command line by checking the Add python.exe to PATH checkbox.

Step 3: Running the Executable Installer

After completing the setup. Python will be installed on your Windows system. You will see a successful message.

Step 4: Verify the Python Installation in Windows

Close the window after successful installation of Python. You can check if the installation of Python was successful by using either the command line or the Integrated Development Environment (IDLE), which you may have installed. To access the command line, click on the Start menu and type “cmd” in the search bar. Then click on Command Prompt.python --version



How to install Python on Windows? (OFFLINE MODE)

Python >>> Downloads >>> Windows

Python Releases for Windows

- [Latest Python 3 Release - Python 3.11.3](#)

Stable Releases

- [Python 3.10.11 - April 5, 2023](#)
Note that Python 3.10.11 cannot be used on Windows 7 or earlier.
 - Download [Windows embeddable package \(32-bit\)](#)
 - Download [Windows embeddable package \(64-bit\)](#)
 - Download [Windows help file](#)
 - Download [Windows installer \(32-bit\)](#)

Python 3.10.11 (64-bit) Setup

Install Python 3.10.11 (64-bit)

Select **Install Now** to install Python with default settings, or choose **Customize** to enable or disable features.

Install Now
C:\Users\ashub\AppData\Local\Programs\Python\Python310

Includes IDLE, pip and documentation
Creates shortcuts and file associations

→ **Customize installation**
Choose location and features

☒ Use admin privileges when installing py.exe
☒ Add python.exe to PATH

Cancel

Python 3.10.11 (64-bit) Setup

Setup Progress

Installing:

Python 3.10.11 Standard Library (64-bit)

Cancel

Python 3.10.11 (64-bit) Setup

Setup was successful

New to Python? Start with the [online tutorial](#) and [documentation](#). At your terminal, type "py" to launch Python, or search for Python in your Start menu.

See [what's new](#) in this release, or find more info about [using Python on Windows](#).

Disable path length limit
Changes your machine configuration to allow programs, including Python, to bypass the 260 character "MAX_PATH" limitation.

Close



PYTHON IDE ONLINE MODE STEP 1 & STEP 2

About 1,100,000 results



Python

Programming language

Updates

Download

Courses

Frameworks



Programiz

<https://www.programiz.com/python-programming/online-compiler>

[Online Python Compiler \(Interpreter\) - Programiz](https://www.programiz.com/python-programming/online-compiler)

Write and run your Python code using our online compiler. Enjoy additional features like code sharing, dark mode, and support for multiple programming languages.

Online R Compiler

Enjoy additional features like code sharing, dark mode, and support for multiple programming languages. {} Online Python Compiler Online R Compiler Online SQL Editor Online HTML/CSS ...

[See results only from programiz.com](#)

People also search for

python mode of list

types of mode in python

The screenshot shows the Programiz Python Online Compiler interface. At the top, there's a navigation bar with the URL <https://www.programiz.com/python-programming/online-compiler/>. Below the navigation bar is a promotional banner for "BLACK FRIDAY SALE" offering a 60% discount on Programiz PRO. The main header features the Programiz logo and the text "Python Online Compiler". A large blue banner advertises "Premium Coding Courses by Programiz". The interface is divided into two main sections: a code editor on the left and an output panel on the right. The code editor shows a file named "main.py" with the code `1 print("this is my online ide for Python")`. Above the code editor are icons for toggling dark mode, a share icon, and a "Run" button. The output panel displays the result of the code execution: "this is my online ide for Python" followed by "=== Code Execution Successful ===".



Google Colab STEP 1 & STEP2

Microsoft Bing

google colab

SEARCH COPILOT IMAGES VIDEOS

About 984,000 results

 Google Colab
<https://colab.research.google.com>

Google Colab

What is Colab? Colab, or "Colaboratory", allows you to run code in a virtual machine with access to GPUs and TPUs. Zero configuration required. Access to GPUs and TPUs is available for free.

Notebook

Colab notebooks allow you to combine executable code and rich text in a single document.

<https://colab.research.google.com/drive/1H66pA1eXvIBsTDC...>

 Untitled0.ipynb ☆

File Edit View Insert Runtime Tools Help

+ Code + Text

 `print("hello worlds***")`

 hello worlds***



Enough to Understand the Code

Indentation matters to code meaning

Block structure indicated by indentation

First assignment to a variable creates it

Variable types don't need to be declared.

Python figures out the variable types on its own.

Assignment is = and comparison is ==

For numbers + - * / % are as expected

Special use of + for string concatenation and % for string formatting (as in C's printf)

Logical operators are words (and, or, not) not symbols

The basic printing command is print



Basic Datatypes

Integers (default for numbers)

```
z = 5 / 2 # Answer 2, integer division
```

Floats

```
x = 3.456
```

Strings

- Can use “” or “” to specify with “abc” == ‘abc’
- Unmatched can occur within the string: “matt’s”
- Use triple double-quotes for multi-line strings or strings than contain both ‘ and “ inside of them:
“““a‘b“c”””



Comments

- Start comments with `#`, rest of line is ignored
- Can include a “documentation string” as the first line of a new function or class you define
- Development environments, debugger, and other tools use it: it’s good style to include one

```
def fact(n):
```

```
    """
```

```
    fact(n) assumes n is a positive integer and returns factorial of n.
```

```
    """
```

```
    assert(n>0)
```

```
        return 1 if n==1 else n*fact(n-1)
```



Naming Rules

Names are case sensitive and cannot start with a number. They can contain letters, numbers, and underscores.

`bob Bob _bob _2_bob_ bob_2 BoB`

There are some reserved words:

`and, assert, break, class, continue, def, del, elif, else, except, exec,
finally, for, from, global, if, import, in, is, lambda, not, or, pass, print,
raise, return, try, while`



Naming conventions

The Python community has these recommend-ed naming conventions

joined_lower for functions, methods and, attributes

joined_lower or ALL_CAPS for constants

StudlyCaps for classes

camelCase only to conform to pre-existing conventions

Attributes: interface, _internal, __private



Assignment

You can assign to multiple names at the same time

```
>>> x, y = 2, 3
```

```
>>> x
```

```
2
```

```
>>> y
```

```
3
```

This makes it easy to swap values

```
>>> x, y = y, x
```

Assignments can be chained

```
>>> a = b = x = 2
```




VARIABLE

Python Variable is containers that store values. Python is not “statically typed”.

Rules for Python variables

- A Python variable name must start with a letter or the underscore character.
- A Python variable name cannot start with a number.
- A Python variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
- Variable in Python names are case-sensitive (name, Name, and NAME are three different variables).
- The RESERVED WORD in Python cannot be used to name the variable in Python.



PRACTICAL 1

PROGRAM TO DIFFERENTIATE DATA TYPE

```
File Edit View Insert Runtime Tools Help
+ Code + Text

# An integer assignment
age = 25

# A floating point
salary = 80000

# A string
name = "Poonam Rana"

print(age)
print(salary)
print(name)
```

25
80000
Poonam Rana

PRACTICAL-2

PROGRAM TO USE OF VAR & EXPRESSIONS

```
File Edit View Insert Runtime Tools Help All changes saved
Code + Text

# VAR & Constant Expressions
x = 15 + 1.3

print(x)

# VAR & Arithmetic Expressions
x = 40
y = 12

add = x + y
sub = x - y
pro = x * y
div = x / y

print(add)
print(sub)
print(pro)
print(div)

# VAR & Integral Expressions
a = 13
b = 12.0

c = a + int(b)
print(c)

# VAR & Floating Expressions
a = 13
b = 5

c = a / b
print(c)
```

16.3
52
28
480
3.3333333333333335
25
2.6



PRACTICAL 3

CALCULATION OF SIMPLE INTEREST

File Edit View Insert Runtime Tools Help Last edited

+ Code + Text

```
[ ] p=int(input("enter principal amt==="))  
    r=float(input("enter rate==="))  
    t=int(input("enter duration==="))  
    si=(p*r*t)/100  
    print(si)
```

```
⇒ enter principal amt===10000  
   enter rate===7.1  
   enter duration===2  
   1420.0
```

Q1) Can you Swap the value of a variable to another variable?



Arithmetic vs Bitwise AND

1) Arithmetic vs Bitwise AND

```
result_1 = 5 + 2 & 3  
print(result_1)
```

2. NOT, AND, or Trap

```
result_1 = True or False and not  
True  
print(result_1)
```

3.

```
result_with = (5 + 2) * (3 | 1)  
# 7 * (3 | 1) -> 7 * 3 -> 21
```

Operator Precedence

Parentheses ()
Arithmetic operators *, / %, -
Bitwise NOT -
Bitwise <<, >>
Bitwise AND
Bitwise XOR
Bitwise OR
Logical operators
not, and, or



Sequence Types

1. Tuple: ('john', 32, [CMSC])
 - A simple *immutable* ordered sequence of items
 - Items can be of mixed types, including collection types
2. Strings: "John Smith"
Immutable
Conceptually very much like a tuple
3. List: [1, 2, 'john', ('up', 'down')]
Mutable ordered sequence of items of mixed types



- All three sequence types (tuples, strings, and lists) share much of the same syntax and functionality.
- Key difference:
- **Tuples and strings are *immutable* Lists are *mutable***
- The operations shown in this section can be applied to *all* sequence types
- **most examples will just show the operation performed on one**

Forward direction indexing				
0	1	2	3	4
List				
MON	TUE	WED	THU	FRI
-5	-4	-3	-2	-1
Backward direction indexing				



Define Sequences

Define tuples using parentheses and commas

```
>>> tu = (23, 'abc', 4.56, (2,3), 'def')
```

Define lists are using square brackets and commas

```
>>> li = ["abc", 34, 4.34, 23]
```

Define strings using quotes (" , ' , or """").

```
>>> st = "Hello World"
```

```
>>> st = 'Hello World'
```

```
>>> st = """This is a multi-line  
string that uses triple quotes."""
```



Access individual members of a tuple, list, or string using square bracket “array” notation

Note that all are 0 based...

```
tu = (23, 'abc', 4.56, (2,3), 'def')  
tu[1]      # Second item in the tuple.  
'abc'
```

```
li = ["abc", 34, 4.34, 23]  
li[1]      # Second item in the list.  
34
```

```
st = "Hello World"  
st[1]      # Second character in string.  
'e'
```



Positive and negative indices

```
t = (23, 'abc', 4.56, (2, 3), 'def')
```

Positive index: count from the left, starting with 0

```
t[1]
```

```
'abc'
```

Negative index: count from right, starting with -1

```
t[-3]
```

```
4.56
```



Slicing: return copy of a subset

```
t = (23, 'abc', 4.56, (2, 3), 'def')
```

Return a copy of the container with a subset of the original members. Start copying at the first index, and stop copying before second.

```
t[1:4]  
( 'abc', 4.56, (2, 3) )
```

Negative indices count from end

```
t[1:-1]  
( 'abc', 4.56, (2, 3) )
```




Slicing: return copy of a =subset

```
t = (23, 'abc', 4.56, (2, 3), 'def')
```

Omit first index to make copy starting from beginning of the container

```
t[:2]
```

```
(23, 'abc')
```

Omit second index to make copy starting at first index and going to end

```
t[2:]
```

```
(4.56, (2, 3), 'def')
```



Copying the Whole Sequence

[:] makes a *copy* of an entire sequence

```
t[:]
```

```
(23, 'abc', 4.56, (2,3), 'def')
```

Note the difference between these two lines for mutable sequences

```
l2 = l1 # Both refer to 1 ref,  
        # changing one affects
```

both

```
l2 = l1[:] # Independent copies, two  
refs
```



The 'in' Operator

Boolean test whether a value is inside a container:

```
t = [1, 2, 4, 5]
```

```
3 in t
```

```
False
```

```
4 in t
```

```
True
```

```
4 not in t
```

```
False
```

For strings, tests for substrings

```
a = 'abcde'
```

```
'c' in a
```

```
True
```

```
'cd' in a
```

```
True
```

```
'ac' in a
```

```
False
```

Be careful: the in keyword is also used in the syntax of for loops and list comprehensions



The + Operator in sequences

The + operator produces a *new* tuple, list, or string whose value is the concatenation of its arguments.

```
(1, 2, 3) + (4, 5, 6)  
(1, 2, 3, 4, 5, 6)
```

```
[1, 2, 3] + [4, 5, 6]  
[1, 2, 3, 4, 5, 6]
```

```
"Hello" + " " + "World"  
'Hello World'
```



The * Operator

The * operator produces a *new* tuple, list, or string that “repeats” the original content.

```
>>> (1, 2, 3) * 3  
(1, 2, 3, 1, 2, 3, 1, 2, 3)
```

```
>>> [1, 2, 3] * 3  
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

```
>>> "Hello" * 3  
'HelloHelloHello'
```




Mutability: Tuples vs. Lists

Lists are mutable

```
li = ['abc', 45, 4.34, 23]
```

```
li[1] = 45
```

```
li
```

```
['abc', 45, 4.34, 23]
```

We can change lists *in place*.

Name *li* still points to the same memory reference when we're done.



Tuples are immutable

```
t = (23, 'abc', 4.56, (2,3), 'def')  
t[2] = 3.14
```

```
Traceback (most recent call last):  
  File "<pyshell#75>", line 1, in -toplevel-  
    tu[2] = 3.14  
TypeError: object doesn't support item assignment
```

You can't change a tuple.

You can make a fresh tuple and assign its reference to a previously used name.

```
t = (23, 'abc', 3.14, (2,3), 'def')
```

The immutability of tuples means they're faster than lists.



Operations on Lists Only

```
>>> li = [1, 11, 3, 4, 5]
```

```
>>> li.append('a') # Note the method  
syntax
```

```
>>> li  
[1, 11, 3, 4, 5, 'a']
```

```
>>> li.insert(2, 'i')
```

```
>>> li  
[1, 11, 'i', 3, 4, 5, 'a']
```



The extend method vs +

+ creates a fresh list with a new memory ref
extend operates on list `li` in place.

```
>>> li.extend([9, 8, 7])
>>> li
[1, 2, 'i', 3, 4, 5, 'a', 9, 8, 7]
```

Potentially confusing:

extend takes a list as an argument.

append takes a singleton as an argument.

```
>>> li.append([10, 11, 12])
>>> li
[1, 2, 'i', 3, 4, 5, 'a', 9, 8, 7, [10, 11, 12]]
```



1. Create List

Create an empty list or a pre-populated

```
list my_list = [10, 20, 30]
```

2. Append (Adds a single element to the very end of the)

```
list my_list.append(40)
```

```
# my_list is now [10, 20, 30, 40]
```

3. Remove / Pop

```
my_list.remove(20)
```

```
# my_list is now [10, 30, 40]
```

```
popped_val = my_list.pop(0)
```




4. Clear (Removes every item, leaving the list completely empty)

```
temp_list = [1, 2, 3]  
temp_list.clear()
```

5. Index

```
temp_list.index(2)
```

6. sort

```
nums.sort()
```

7. Copy

```
original = [1, 2, 3] cloned = original.copy()
```



Merge Lists

1. Using + operator

```
list1 = [1, 2]
```

```
list2 = [3, 4]
```

```
merged = list1 + list2
```

```
# Output: [1, 2, 3, 4]
```

2. Using extend function

```
list1 = [1, 2]
```

```
list2 = [3, 4]
```

```
list1.extend(list2)
```

```
# list1 is now: [1, 2, 3, 4]
```



Merge to 2D list

1. Separate Rows Using []

```
list1 = [1, 2, 3]
```

```
list2 = ['a', 'b', 'c']
```

```
matrix = [list1, list2] # Output: [[1, 2, 3], ['a', 'b', 'c']]
```

2. Using zip() function

```
list1 = [1, 2, 3]
```

```
list2 = ['a', 'b', 'c']
```

```
# zip() pairs them as tuples,
```

```
list comprehension turns them into lists
```

```
matrix = [list(item) for item in zip(list1, list2)]
```

```
# Output: [[1, 'a'], [2, 'b'], [3, 'c']]
```



Tuple details

The comma is the tuple creation operator, not parens

```
1,
```

```
(1,)
```

Python shows parens for clarity (best practice)

```
(1,)
```

```
(1,)
```

Don't forget the comma!

```
(1)
```

```
1
```

Trailing comma only required for singletons others

Empty tuples have a special syntactic form

```
()
```

```
()
```

```
tuple()
```

```
()
```



Initialization

1. **Wrong:** This creates an integer, not a tuple!

```
not_a_tuple = (5)
```

```
print(type(not_a_tuple))  
# Output: <class 'int'>
```

2. **Right:** The trailing comma defines the

```
tuplea_real_tuple = (5,)
```

```
print(type(a_real_tuple))  
# Output: <class 'tuple'>
```

3. **Right:** No parentheses, but the comma makes it a

```
tupleimplicit_tuple = 1, 2, 3
```

```
print(type(implicit_tuple))  
Output: <class 'tuple'>
```




Mutation

1. A tuple containing an integer and a list

```
tricky_tuple = (1, [2, 3])  
# tricky_tuple[0] = 99
```

(This will crash (cannot change the tuple structure))

2. It works if We are modifying the list inside the tuple

```
tricky_tuple[1].append(4)  
print(tricky_tuple)
```

```
# Output: (1, [2, 3, 4])
```



Practical work

Q1 print the count of 2s in this tuple

Q2 print the index of 3 in this tuples

```
nums = (1, 2, 3, 2, 4)
```

Q3 Unpack this tuple into two separate tuples

```
coordinates = (10, 20)
```

Q4 Join two tuples

```
t1 = (1, 2) t2 = (3, 4)
```



Summary: Tuples vs. Lists

Lists slower but more powerful than tuples

Lists can be modified, and they have lots of handy operations and methods

Tuples are immutable and have fewer features

To convert between tuples and lists use the `list()` and `tuple()` functions:

```
li = list(tu)
```

```
tu = tuple(li)
```



ANY
QUERIES??



Thankyou!!!!

